Sorting Algorithms and Practice Questions

1. Merge Sort

What is Merge Sort?

Merge Sort is a divide-and-conquer algorithm that divides the array into two halves, recursively sorts them, and then merges the sorted halves back together.

How Merge Sort Works:

1. **Divide**: Split the array into two halves
2. **Conquer**: Recursively sort both halves
3. **Combine**: Merge the two sorted halves

Time Complexity:

- Best Case: O(n log n)
- Average Case: O(n log n)
- Worst Case: O(n log n)

Space Complexity: O(n)

Java Implementation:

java

```java
public class MergeSort {

    public static void mergeSort(int[] arr, int left, int right) {
        if (left < right) {
            int mid = left + (right - left) / 2;

            // Sort first and second halves
            mergeSort(arr, left, mid);
            mergeSort(arr, mid + 1, right);

            // Merge the sorted halves
            merge(arr, left, mid, right);
        }
    }

    public static void merge(int[] arr, int left, int mid, int right) {
        // Find sizes of two subarrays to be merged
        int n1 = mid - left + 1;
        int n2 = right - mid;

        // Create temp arrays
        int[] leftArray = new int[n1];
        int[] rightArray = new int[n2];

        // Copy data to temp arrays
        for (int i = 0; i < n1; i++) {
            leftArray[i] = arr[left + i];
        }
        for (int j = 0; j < n2; j++) {
            rightArray[j] = arr[mid + 1 + j];
        }

        // Merge the temp arrays
        int i = 0, j = 0, k = left;

        while (i < n1 && j < n2) {
            if (leftArray[i] <= rightArray[j]) {
                arr[k] = leftArray[i];
                i++;
            } else {
                arr[k] = rightArray[j];
                j++;
            }
```

```
            k++;
        }

        // Copy remaining elements
        while (i < n1) {
            arr[k] = leftArray[i];
            i++;
            k++;
        }

        while (j < n2) {
            arr[k] = rightArray[j];
            j++;
            k++;
        }
    }
}
```

Example Usage:

```java
public static void main(String[] args) {
    int[] arr = {12, 11, 13, 5, 6, 7};
    System.out.println("Original array: " + Arrays.toString(arr));

    mergeSort(arr, 0, arr.length - 1);
    System.out.println("Sorted array: " + Arrays.toString(arr));
}
```

2. Quick Sort

What is Quick Sort?

Quick Sort is a divide-and-conquer algorithm that picks a pivot element and partitions the array around the pivot, placing smaller elements before it and larger elements after it.

How Quick Sort Works:

1. **Choose a pivot** element from the array
2. **Partition** the array so elements smaller than pivot come before it, and larger elements come after
3. **Recursively** apply the same process to the sub-arrays

Time Complexity:

- Best Case: O(n log n)
- Average Case: O(n log n)
- Worst Case: $O(n^2)$

Space Complexity: O(log n)

Java Implementation:

```java
public class QuickSort {

    public static void quickSort(int[] arr, int low, int high) {
        if (low < high) {
            // Partition the array and get pivot index
            int pivotIndex = partition(arr, low, high);

            // Recursively sort elements before and after partition
            quickSort(arr, low, pivotIndex - 1);
            quickSort(arr, pivotIndex + 1, high);
        }
    }

    public static int partition(int[] arr, int low, int high) {
        // Choose the rightmost element as pivot
        int pivot = arr[high];
        int i = low - 1; // Index of smaller element

        for (int j = low; j < high; j++) {
            // If current element is smaller than or equal to pivot
            if (arr[j] <= pivot) {
                i++;
                swap(arr, i, j);
            }
        }

        // Place pivot in correct position
        swap(arr, i + 1, high);
        return i + 1;
    }

    public static void swap(int[] arr, int i, int j) {
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
}
```

Example Usage:

```java
public static void main(String[] args) {
    int[] arr = {10, 7, 8, 9, 1, 5};
    System.out.println("Original array: " + Arrays.toString(arr));

    quickSort(arr, 0, arr.length - 1);
    System.out.println("Sorted array: " + Arrays.toString(arr));
}
```

3. Count Sort

What is Count Sort?

Count Sort is a non-comparison based sorting algorithm that works by counting the occurrences of each distinct element in the array. It's efficient when the range of input data is small.

How Count Sort Works:

1. **Find the range** of input data (minimum and maximum values)
2. **Create a count array** to store the count of each unique element
3. **Count occurrences** of each element
4. **Modify the count array** to store actual positions
5. **Build the output array** using the count array

Time Complexity: O(n + k) where k is the range of input

Space Complexity: O(k)

Java Implementation:

java

```java
public class CountSort {

    public static void countSort(int[] arr) {
        // Find the maximum element to determine range
        int max = findMax(arr);
        int min = findMin(arr);
        int range = max - min + 1;

        // Create count array
        int[] count = new int[range];
        int[] output = new int[arr.length];

        // Store count of each element
        for (int i = 0; i < arr.length; i++) {
            count[arr[i] - min]++;
        }

        // Change count[i] to actual position of element in output array
        for (int i = 1; i < count.length; i++) {
            count[i] += count[i - 1];
        }

        // Build output array
        for (int i = arr.length - 1; i >= 0; i--) {
            output[count[arr[i] - min] - 1] = arr[i];
            count[arr[i] - min]--;
        }

        // Copy output array to original array
        for (int i = 0; i < arr.length; i++) {
            arr[i] = output[i];
        }
    }

    public static int findMax(int[] arr) {
        int max = arr[0];
        for (int i = 1; i < arr.length; i++) {
            if (arr[i] > max) {
                max = arr[i];
            }
        }
        return max;
    }
```

```java
    public static int findMin(int[] arr) {
        int min = arr[0];
        for (int i = 1; i < arr.length; i++) {
            if (arr[i] < min) {
                min = arr[i];
            }
        }
        return min;
    }
}
```

Example Usage:

```java
public static void main(String[] args) {
    int[] arr = {4, 2, 2, 8, 3, 3, 1};
    System.out.println("Original array: " + Arrays.toString(arr));

    countSort(arr);
    System.out.println("Sorted array: " + Arrays.toString(arr));
}
```

When to Use Count Sort:

- When the range of input data (k) is not significantly larger than the number of elements (n)
- When you need a stable sorting algorithm
- When dealing with integers in a known range

Quiz Questions

1. What is the worst-case time complexity of Quick Sort? a) O(n log n) b) $O(n^2)$ c) O(n) d) O(log n)

Answer: b) $O(n^2)$

2. Which sorting algorithm uses the divide-and-conquer approach and always has O(n log n) time complexity? a) Quick Sort b) Count Sort c) Merge Sort d) Bubble Sort

Answer: c) Merge Sort

3. Count Sort is most efficient when: a) The array size is very large b) The range of input values is small compared to the number of elements c) The array is already sorted d) The array contains only negative

numbers

Answer: b) The range of input values is small compared to the number of elements

4. What is the space complexity of Merge Sort? a) O(1) b) O(log n) c) O(n) d) O(n^2)

Answer: c) O(n)

5. In Quick Sort, what is the purpose of the partition function? a) To divide the array into equal halves b) To find the maximum element c) To rearrange elements so smaller ones are before the pivot and larger ones are after d) To merge two sorted arrays

Answer: c) To rearrange elements so smaller ones are before the pivot and larger ones are after

---

Practice Coding Questions

1. Merge Sort Modification Modify the merge sort algorithm to sort an array of strings in alphabetical order. Write the complete implementation including the merge function.

2. Quick Sort with Random Pivot Implement Quick Sort where instead of always choosing the last element as pivot, you randomly select a pivot element from the array. Include the random pivot selection logic.

3. Count Sort for Characters Write a Count Sort implementation that can sort an array of lowercase English characters (a-z). The function should take a char array as input and sort it in ascending order.

4. Hybrid Sorting Algorithm Create a sorting function that uses Quick Sort for large arrays (size > 50) and Merge Sort for smaller arrays. The function should automatically choose the appropriate algorithm based on array size.

5. Count Sort with Negative Numbers Extend the Count Sort algorithm to handle arrays that contain both positive and negative integers. Test your implementation with an array like [-5, -1, 0, 3, -2, 4, 1].